Université du Québec à Montréal – UQAM

Comprendre le RAG — Vers une IA qui interroge vos documents intelligemment

Une exploration technique et appliquée du modèle
Retrieval-Augmented Generation (RAG)

Michel Héon, Ph. D.
Conseiller en architecture d'entreprise
heon.michel@uqam.ca
07/05/2025

Document de travail – Version mai 2025

Résumé

Ce document de travail propose une introduction stratégique et technique au modèle Retrieval-Augmented Generation (RAG), une approche d'intelligence artificielle qui transforme les documents internes d'une organisation en assistant conversationnel intelligent. À l'intersection de la recherche sémantique et des grands modèles de langage (LLM), le RAG permet de poser des questions en langage naturel et d'obtenir des réponses fiables, contextualisées et sourcées.

Destiné aux décideurs, architectes TI et responsables de l'innovation, ce guide vulgarise les étapes clés du pipeline RAG — indexer, interroger, générer — et en illustre les bénéfices concrets pour les organisations. Il présente également une architecture mutualisée mise en œuvre à l'UQAM pour créer des assistants spécialisés par domaine, déployés sur Azure.

Mots-clés :

RAG, IA générative, Recherche sémantique, Grands modèles de langage (LLM), Indexation vectorielle, Assistant conversationnel, Azure AI Search, Prompt engineering, Architecture mutualisée, Transformation numérique

Table des matières

Résumé corporatif 1
Genèse du document 1
Introduction 2
1. Comprendre le RAG : synthèse contextuelle par interrogation de bases de connaissances 3
2. Transformer le langage en données numériques 4
3. Espace vectoriel et recherche sémantique 6
Étape 1 — Construire l'espace sémantique 6
Étape 2 — Mesurer la similarité sémantique 7
4. Génération enrichie : étapes du raisonnement automatique par le LLM 7
Étape 2 — Vectoriser : donner un sens aux jetons 8
Étape 3 — Comprendre : analyser l'intention 9
Étape 4 — Prédire : anticiper le mot suivant 9
Étape 5 — Générer : construire la réponse, mot par mot 9
Étape 6 — Détokeniser : reconstruire une réponse lisible 9
5. Architecture détaillée du RAG : du vecteur à la réponse contextualisée 9
Étapes techniques de l'indexation, de l'interrogation et de la génération 9
Phase 1 — Indexation : transformer les documents en empreintes sémantiques 10
Phase 2 — Interrogation : chercher la signification dans l'espace vectoriel 10
Phase 3 — Génération : construire une réponse contextualisée 11
6. Une architecture mutualisée pour déployer des assistants RAG par domaine ... 11
Phase 1 — Indexer : transformer les documents en empreintes sémantiques 12
Phase 2 — Interroger : chercher la signification dans l'espace vectoriel 13
Phase 3 — Générer : construire une réponse contextualisée 13
Une spécialisation fonctionnelle sur une base mutualisée 13
Pilotage technique et gouvernance 14
Récapitulatif des composantes alignées sur le pipeline RAG 14
Conclusion 14

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

Lexique 15

Table des figures

Figure 1: Fonctionnement d'un système RAG : de la question à la réponse contextualisée 3
Figure 2: Tokeniser, vectoriser, représenter : les bases de la recherche sémantique . 5
Figure 3: Similitude cosinus et proximité sémantique 6
Figure 4: Le raisonnement automatique d'un modèle de langage, étape par étape ... 8
Figure 5: RAG en trois phases : indexer, interroger, générer 10
Figure 6 : UQAM-GPT Multi services — Architecture par mutualisation des ressources 12

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

Résumé corporatif

Ce document propose une vulgarisation stratégique du modèle Retrieval-Augmented Generation (RAG), adapté aux décideurs, gestionnaires de projets et responsables de l'innovation numérique. Il explique comment le RAG permet d'interroger un corpus documentaire interne en langage naturel, et d'obtenir des réponses fiables, contextualisées et sourcées grâce à l'intelligence artificielle générative.

Axé sur les bénéfices concrets pour les organisations – gain de temps, réduction des erreurs, valorisation du savoir – le document détaille les étapes techniques essentielles : indexation vectorielle, recherche sémantique, et génération enrichie. Il illustre aussi comment une architecture RAG peut être intégrée dans les outils numériques existants, comme Teams, SharePoint ou des systèmes internes, en mode pilote ou mutualisé par domaine métier.

Ce résumé, qui vise à soutenir une prise de décision éclairée, offre une vue d'ensemble claire et actionnable du RAG en tant que levier de transformation numérique orientée vers les connaissances.

Genèse du document

Ce document a été préparé dans le cadre **des Cafés de l'innovation**, organisée à l'Université du Québec à Montréal le **2 mai 2025**. L'événement, visait à présenter des innovations concrètes en intelligence artificielle appliquée à la gestion des connaissances.

La présentation intitulée **« Comprendre le RAG – Vers une IA qui interroge vos documents intelligemment »** a permis d'introduire les principes, bénéfices et mécanismes techniques du *Retrieval-Augmented Generation (RAG)*. Elle s'adressait à un public de **décideurs, architectes, chercheurs et professionnels TI**, intéressés par les enjeux de contextualisation, de maintenance et d'architecture dans les systèmes d'IA.

Le présent texte reprend et approfondit les éléments clés abordés lors de cette intervention.

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

Introduction

Dans bien des organisations, des montagnes de documents s'accumulent au fil des ans — rapports, procédures internes, courriels, pages wiki — formant un gisement d'information souvent sous-exploité. Et pourtant, lorsque survient une question précise, il demeure étonnamment difficile d'obtenir une réponse rapide, fiable, et contextualisée. On tâtonne, on interroge ses collègues, on fouille… parfois en vain. L'information existe, mais elle reste dispersée, invisible, hors d'atteinte.

C'est précisément à ce défi que répond l'approche dite *Retrieval-Augmented Generation* (RAG). Elle articule deux capacités complémentaires : d'un côté, un moteur de recherche sémantique qui identifie le sens des mots plutôt que leur simple occurrence ; de l'autre, un grand modèle de langage (LLM) capable de rédiger une réponse claire, structurée et adaptée au ton de la demande.

Sur le plan technique, le RAG commence par transformer les documents de l'organisation — qu'ils soient au format PDF, issus d'un wiki, d'une base SQL ou extrait d'une vidéo — en vecteurs numériques, véritables empreintes sémantiques. Ces vecteurs sont indexés dans une base spécialisée. Lorsqu'un utilisateur pose une question, le système interroge cette base pour en extraire les passages les plus pertinents, construit un prompt enrichi, puis le confie au LLM. En retour, il obtient une réponse synthétique, fiable, sourcée, formulée en langage naturel.

Ce processus, qui s'opère en quelques secondes, change la donne à tous les niveaux de l'organisation :

- Les **décideurs** gagnent en pilotage, en traçabilité et en maîtrise des risques.
- Les **professionnels métier** accèdent enfin à l'information essentielle sans avoir à parcourir des dizaines de pages.
- Les **équipes techniques** disposent d'une architecture modulable, intégrable et observable.
- Les **utilisateurs finaux**, eux, bénéficient d'une interface simple, sans qu'aucune formation ne soit requise.

En somme, le RAG ne se contente pas d'améliorer l'accès à l'information : il transforme un corpus documentaire inerte en un assistant conversationnel, capable de comprendre les questions et d'y répondre avec pertinence, comme le ferait un expert… mais en quelques secondes.

Ce document propose une exploration progressive de cette approche. Il commence par poser les fondements du RAG, puis en décrit le fonctionnement technique, avant d'aborder les modalités concrètes de son intégration dans les systèmes et les processus métier existants.

Michel Héon PhD
heon.miche@uqam.ca
UQAM

1. Comprendre le RAG : synthèse contextuelle par interrogation de bases de connaissances

Principe général et fonctionnement global d'un système "Retrieval-Augmented Generation (RAG) »

Un système RAG repose sur un principe simple, mais puissant : permettre à une personne de formuler une question en langage naturel, et de recevoir en retour une réponse claire, structurée et ancrée dans les documents de son organisation.

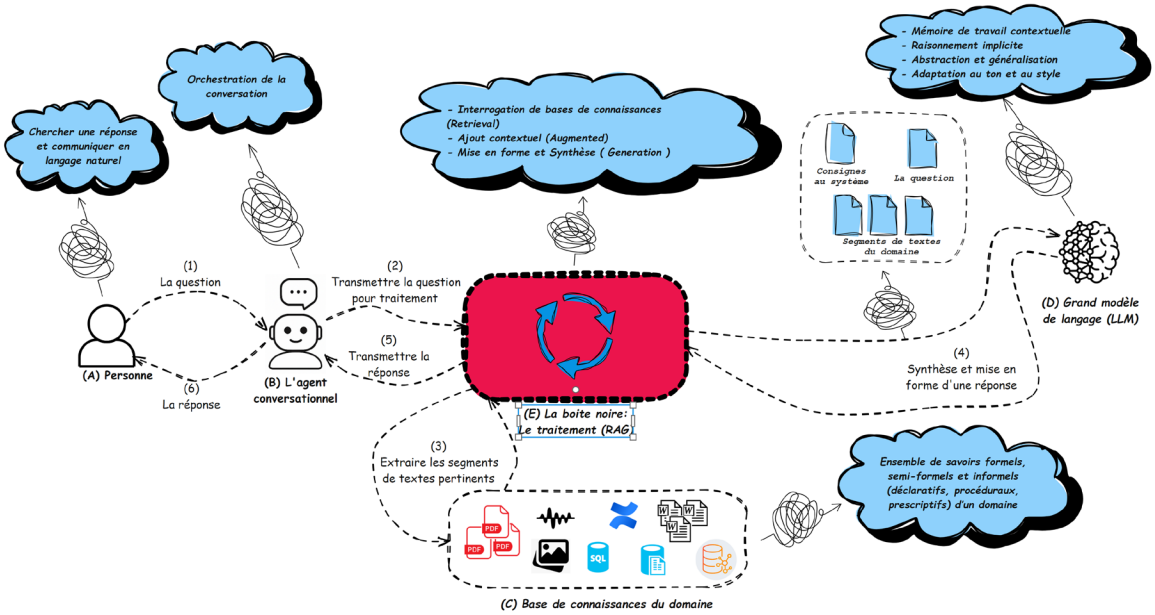

Figure 1: Fonctionnement d'un système RAG : de la question à la réponse contextualisée

Le schéma ci-contre illustre ce cheminement, de la question initiale à la réponse finalisée. Tout commence lorsqu'un utilisateur (A) adresse sa question à un **agent conversationnel** (B), intégré à une interface familière — qu'il s'agisse d'un chatbot, d'un canal Teams ou d'un formulaire web. Cet agent transmet la requête à ce que l'on pourrait appeler la **boîte noire** du système (E), là où s'opère la véritable alchimie du RAG.

À l'intérieur de cette boîte, trois opérations se succèdent :

- **L'extraction d'information pertinente**,
- **L'enrichissement contextuel de la question**,
- **La génération de la réponse**.

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

Pour y parvenir, le système interroge une **base de connaissances vectorielle** (C), construite à partir de documents internes hétérogènes — fichiers PDF, bases de données, wikis, procès-verbaux — qui ont préalablement été transformés en **empreintes sémantiques** grâce à des algorithmes d'« embedding ». À ce stade, il ne s'agit plus de rechercher des mots, mais des **idées** ou des **sens** proches de la question posée.

Le moteur de recherche sémantique identifie alors les extraits les plus significatifs, qui sont ensuite assemblés avec la question et quelques consignes dans un **prompt enrichi**. Ce prompt est ensuite transmis à un **grand modèle de langage** (D), lequel mobilise ses capacités de raisonnement contextuel pour produire une réponse adaptée à la demande.

Celle-ci est renvoyée à l'agent conversationnel (5), puis livrée à l'utilisateur (6) dans un format compréhensible, synthétique, et souvent accompagné de citations ou de sources internes.

Ce processus, rapide et invisible à l'utilisateur, permet d'accéder non seulement à des savoirs formels (comme une clause contractuelle), mais aussi à des contenus plus diffus : pratiques internes, procédures implicites, décisions non codifiées. Il agit comme une passerelle entre l'intuition humaine et la mémoire organisationnelle.

En intégrant cette logique dans leurs outils numériques habituels, les organisations peuvent répondre à un besoin très concret : **fournir un accès instantané, fiable et ciblé à l'information métier**, sans imposer un nouvel apprentissage aux employés. Qu'il s'agisse de vérifier une règle de télétravail, d'appliquer une procédure RH ou de répondre à une question réglementaire, le RAG transforme une masse documentaire souvent inexploitée en **réponses directement opérationnelles**.

Au-delà du gain de temps, cette approche réduit les erreurs d'interprétation, allège la charge des experts internes et révèle la valeur stratégique des connaissances déjà présentes. Le RAG devient alors un **levier de transformation organisationnelle**, au service de l'efficacité, de la fiabilité et du transfert des savoirs.

2. Transformer le langage en données numériques

Tokenisation et vectorisation : les premières étapes de la compréhension automatique du langage.

Pour qu'un système d'intelligence artificielle puisse traiter le langage humain, encore faut-il **le traduire dans une forme qu'il peut manipuler**. Le langage, dans sa richesse naturelle, doit d'abord être **converti en données numériques structurées**, afin de permettre des opérations de comparaison, de classement ou de raisonnement.

Michel Héon PhD
heon.miche@uqam.ca
UQAM

C'est précisément ce que montre la figure ci-contre, qui illustre les étapes allant de la phrase brute à sa représentation vectorielle. Ce parcours s'effectue en quatre temps, répartis sur les points A à F du schéma.

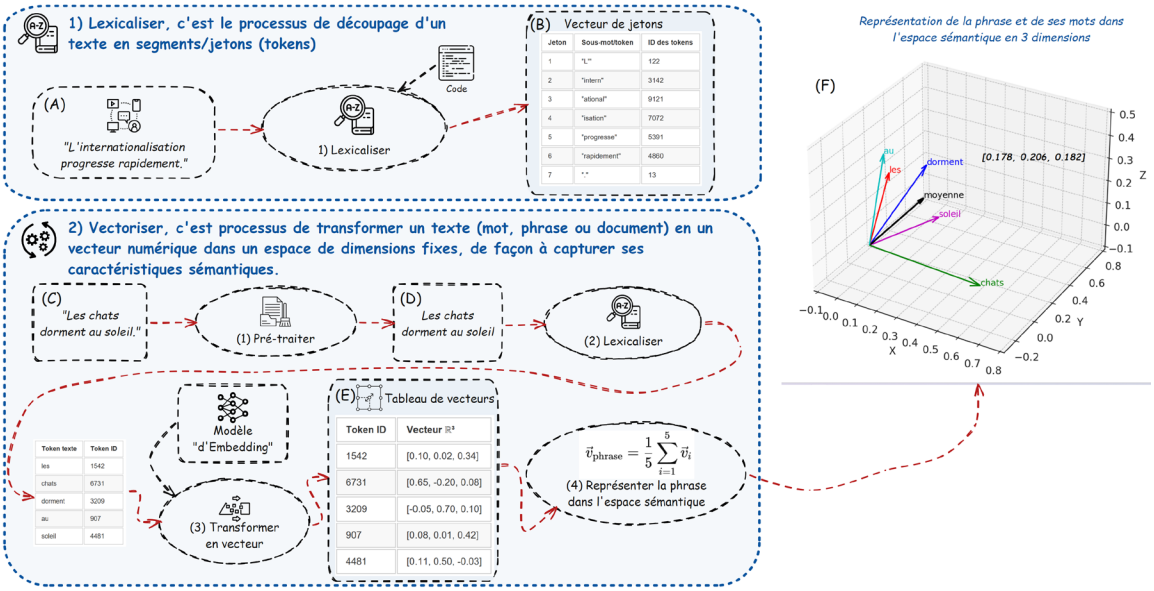

Figure 2: Tokeniser, vectoriser, représenter : les bases de la recherche sémantique

Tout commence par la **lexicalisation** (A), parfois appelée *tokenisation*. L'algorithme découpe la phrase en unités élémentaires appelées *jetons* — qui ne sont pas nécessairement des mots entiers, mais souvent des fragments de mots récurrents (par exemple « intern » ou « alisation »). Chacun de ces jetons est alors associé à un identifiant numérique unique (B). Cette opération transforme un texte lisible en une suite de codes, une étape cruciale vers une compréhension automatisée.

Une nouvelle phrase est ensuite introduite dans le système (C), puis elle suit le même processus : nettoyage, normalisation, découpage (D). À cette étape, le texte est prêt à entrer dans ce que l'on appelle un **modèle d'embedding**, qui convertit chaque jeton en **vecteur sémantique** : une suite de nombres représentant la position du mot dans un espace abstrait (E). Ces coordonnées capturent des **indices de signification** : deux mots proches en sens se retrouvent proches géométriquement.

Enfin, l'ensemble des vecteurs associés à la phrase est moyenné pour obtenir une **empreinte globale** — un point unique qui représente toute la séquence textuelle dans un espace vectoriel (F). Ce point pourra ensuite être comparé à d'autres, permettant des rapprochements sémantiques bien au-delà des simples correspondances lexicales.

En résumé, chaque fois qu'un système RAG ou un assistant conversationnel « comprend » une question, il **suit ce même itinéraire** : découper le texte, encoder le

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

sens, projeter dans un espace mathématique, puis utiliser cette projection pour retrouver l'information pertinente. Sans cette transformation fondamentale — **du mot au vecteur** — aucune forme d'intelligence artificielle ne serait en mesure d'associer efficacement une question à son sens profond.

3. Espace vectoriel et recherche sémantique

Comment les vecteurs permettent de comparer le sens des mots

Pour que l'intelligence artificielle puisse comparer des idées plutôt que des mots, elle doit disposer d'une représentation mathématique du sens. C'est tout l'intérêt de l'espace vectoriel, un espace abstrait dans lequel chaque mot ou fragment de texte est représenté par un vecteur — une suite de nombres — qui capture sa signification dans un contexte donné.

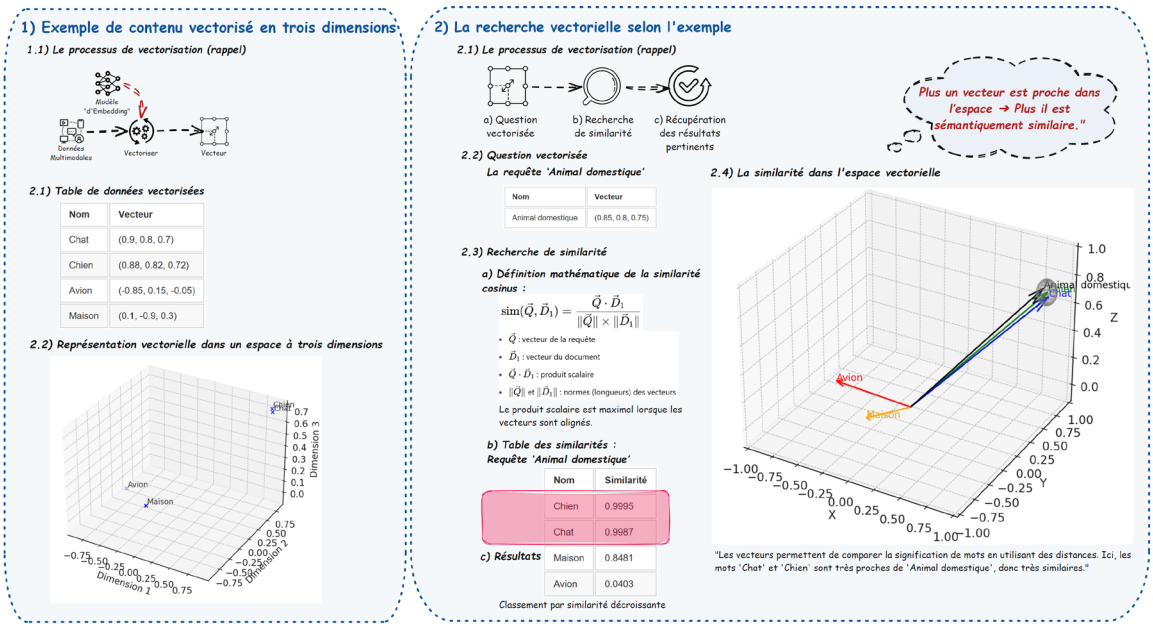

Figure 3: Similitude cosinus et proximité sémantique

La figure montre comment un système de traitement du langage passe d'une liste de mots vectorisés à une recherche sémantique efficace.

Étape 1 — Construire l'espace sémantique

Comme illustré dans la figure 3 (1.1), un modèle d'**embedding** transforme des données multimodales (texte, audio, image) en vecteurs numériques. Dans l'exemple, les mots « Chat », « Chien », « Avion » et « Maison » sont traduits en coordonnées numériques (1.2). Ces vecteurs sont ensuite visualisés dans un **nuage de points** en trois dimensions : les mots proches en sens, comme « Chat » et « Chien

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

», apparaissent géométriquement voisins, tandis que d'autres, comme « Avion », s'en éloignent nettement.

Ce nuage vectoriel constitue un **espace sémantique**, où la distance entre les points reflète une forme de proximité de sens.

Étape 2 — Mesurer la similarité sémantique

Lorsque l'utilisateur saisit une requête — ici « Animal domestique » (2.2) — celle-ci est également vectorisée. Le système calcule ensuite la **similarité cosinus** (2.3a) entre ce vecteur et ceux de tous les mots indexés. Cette mesure indique à quel point deux vecteurs sont alignés : plus l'angle est petit, plus les significations sont proches.

Le système produit un **classement des résultats** (2.3b–c), dans lequel « Chien » et « Chat » obtiennent des scores élevés, indiquant une forte proximité sémantique avec « Animal domestique ». En revanche, « Maison » et « Avion » apparaissent comme moins pertinents.

Une recherche fondée sur le sens, pas sur les mots

Le graphique (2.4) illustre visuellement ce phénomène : la requête et les mots les plus pertinents convergent dans le même voisinage géométrique. C'est cette **proximité dans l'espace vectoriel** qui permet à un système RAG, un assistant vocal ou un moteur sémantique de restituer des résultats cohérents, même si la requête ne contient aucun des mots-clés présents dans les documents.

En résumé, **la recherche vectorielle rend possible l'appariement d'idées**, en dépassant les limites du mot-clé exact. Elle permet à l'IA de reconnaître que « animal domestique » évoque « chien » et « chat », même si ces termes ne sont pas explicitement mentionnés dans la question initiale.

4. Génération enrichie : étapes du raisonnement automatique par le LLM

Étapes techniques de traitement lexical, sémantique et prédictif

Un modèle de langage (LLM) ne répond pas de manière figée. Il **reconstruit une réponse à chaque requête**, en mobilisant un raisonnement basé sur le contexte fourni. Cette section décrit comment il transforme un ensemble de données — question, contexte et consignes — en une réponse naturelle, cohérente et fondée.

Michel Héon PhD
heon.miche@uqam.ca
UQAM

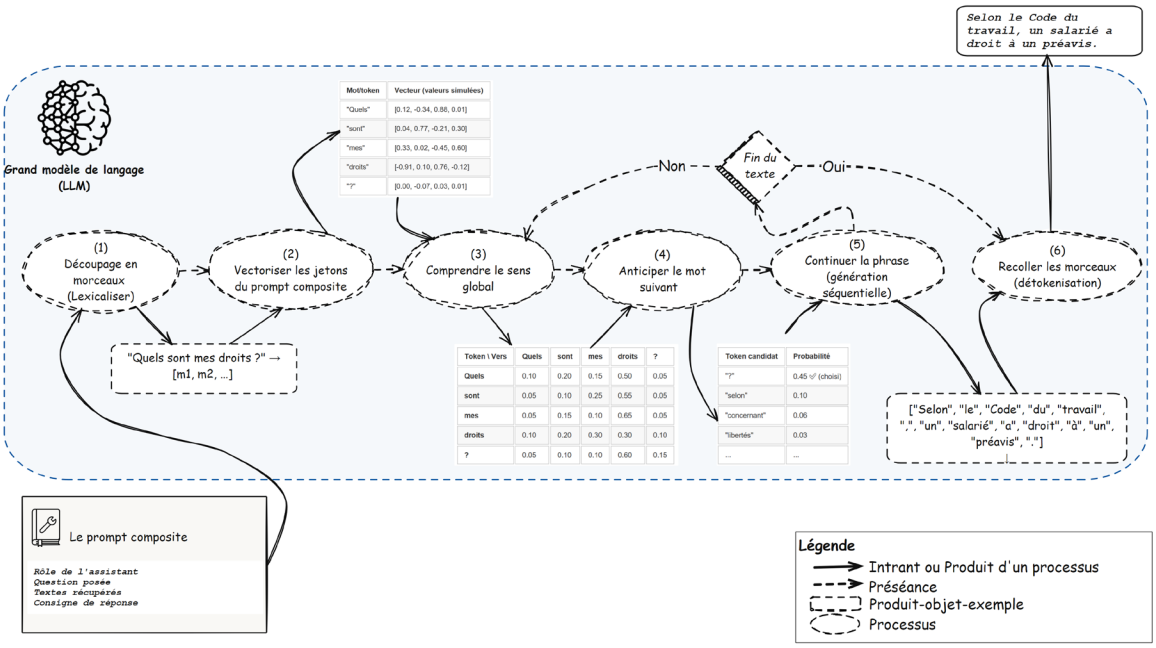

Figure 4: Le raisonnement automatique d'un modèle de langage, étape par étape

Le schéma illustre comment un LLM produit une réponse à partir d'une question en langage naturel, comme « Quels sont mes droits ? » qui se décompose en 6 étapes :

Étape 1 — Lexicaliser : découper le texte en unités manipulables

Tout commence avec le **prompt composite**, une structure qui regroupe :

- la question de l'utilisateur,
- les extraits de documents jugés pertinents,
- et les consignes de génération (style, ton, niveau de détail…).

Le LLM débute son travail par une **lexicalisation** : il découpe l'ensemble du prompt en **jetons** (ou *tokens*), qui sont des segments élémentaires (mots, sous-mots, signes). Par exemple, la phrase « Quels sont mes droits ? » devient une séquence de jetons numériques [m1, m2, …], première étape vers une compréhension automatisée.

Étape 2 — Vectoriser : donner un sens aux jetons

Chaque jeton est ensuite **transformé en vecteur sémantique**. Ces vecteurs permettent de capturer non seulement la signification des mots, mais aussi leur **contexte d'apparition**. Ainsi, le modèle peut différencier le mot « Java » selon qu'il désigne un café ou un langage de programmation.

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

Étape 3 — Comprendre : analyser l'intention

À ce stade, le modèle établit une **représentation cohérente** de l'ensemble du prompt. Il analyse les relations entre les vecteurs, repère les éléments clés de la demande, et **formule une compréhension globale de l'intention de l'utilisateur**. C'est la phase de « mise en contexte ».

Étape 4 — Prédire : anticiper le mot suivant

Le cœur du fonctionnement réside ici. Le LLM **ne choisit pas un mot de manière fixe**, mais prédit une **distribution de probabilités** pour les mots possibles suivants. Par exemple : [« selon » : 42 %, « concernant » : 18 %, « libertés » : 15 %, « ? » : 12 %...]. Le mot ayant la probabilité la plus élevée est sélectionné.

Étape 5 — Générer : construire la réponse, mot par mot

Le modèle procède alors à une génération **séquentielle** : chaque mot produit est réinjecté dans le contexte pour **calculer le suivant**, formant ainsi une chaîne dynamique. Cette boucle continue jusqu'à ce qu'un **signal de fin** soit détecté (souvent un token spécial).

Étape 6 — Détokeniser : reconstruire une réponse lisible

Enfin, les jetons générés sont **convertis en texte lisible** (détokenisation). Le résultat est une phrase naturelle, fluide, et adaptée à la demande initiale. Exemple : *« Selon le Code du travail, un salarié a droit à un préavis. »*

5. Architecture détaillée du RAG : du vecteur à la réponse contextualisée

Étapes techniques de l'indexation, de l'interrogation et de la génération

La figure 5 résume le fonctionnement d'un système RAG en trois grandes phases : **indexer**, **interroger**, **générer**.

Michel Héon PhD
heon.miche@uqam.ca
UQAM

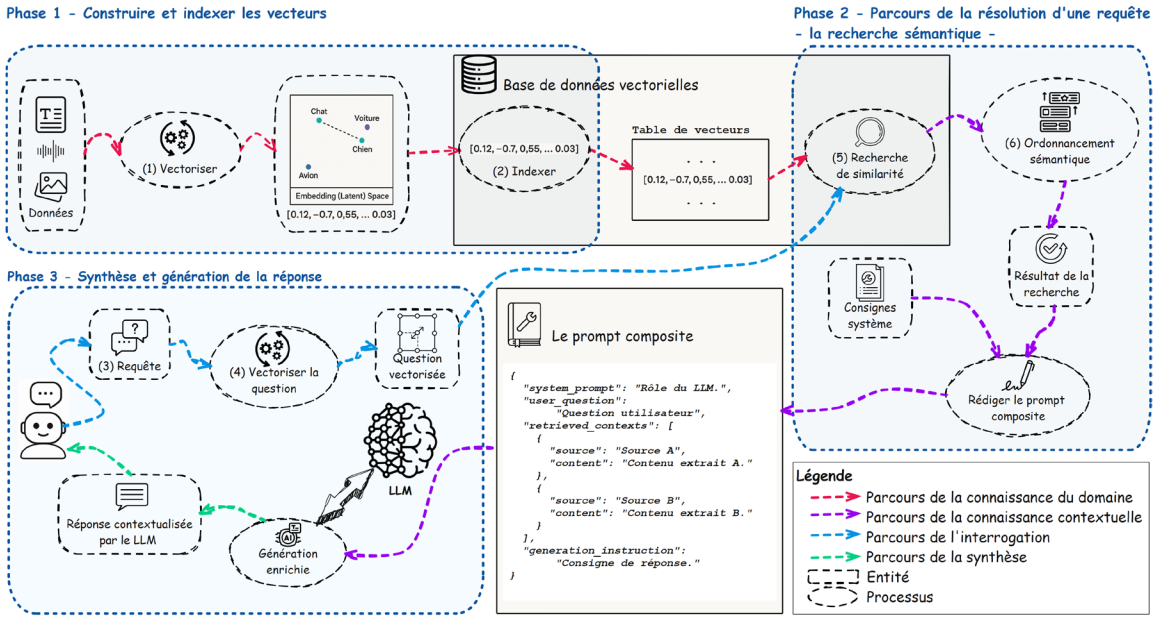

Figure 5: RAG en trois phases : indexer, interroger, générer

Phase 1 — Indexation : transformer les documents en empreintes sémantiques

Tout commence par la constitution d'un espace de connaissances vectoriel. Les documents du domaine — rapports, wikis, bases SQL, fichiers PDF — sont découpés en fragments cohérents (paragraphe, section, entrée de tableau...). Chacun de ces fragments est transformé en vecteur numérique grâce à un modèle d'**embedding**, une sorte de traducteur sémantique qui attribue à chaque fragment une « empreinte » unique dans l'espace vectoriel. Ces vecteurs sont ensuite **indexés** dans une base spécialisée conçue pour la recherche par similarité.

Ce processus, appelé pipeline d'ingestion, ne s'effectue qu'une seule fois lors de la préparation du corpus. Il constitue le **"qui"** et le **"où"** du savoir accessible.

Phase 2 — Interrogation : chercher la signification dans l'espace vectoriel

Lorsqu'un utilisateur pose une question (3), celle-ci est à son tour vectorisée (4) de manière à la représenter dans le même espace sémantique que les documents. Le système lance ensuite une **recherche par similarité** (5), identifiant les fragments les plus proches de la question sur le plan du sens — et non des mots-clés.

Ces résultats sont triés et sélectionnés (6) selon leur **proximité cosinus**, une mesure mathématique de similarité. Le système retient les passages les plus pertinents pour construire la réponse.

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025

Phase 3 — Génération : construire une réponse contextualisée

C'est ici qu'intervient le **prompt composite**, cœur du mécanisme. Ce prompt contient quatre éléments bien structurés :

- le **system_prompt**, qui fixe le rôle et le ton du modèle (ex. : "réponds comme un expert juridique"),
- la **user_question**, parfois reformulée pour plus de clarté,
- les **retrieved_contexts**, c'est-à-dire les extraits de documents pertinents accompagnés de leur source,
- et la **generation_instruction**, qui précise les consignes de réponse (style attendu, niveau de détail, format, citations…).

Ce prompt normalisé est ensuite transmis au **modèle de langage (LLM)**, qui raisonne sur le contenu fourni et rédige une réponse synthétique, ancrée dans les documents et formulée en langage naturel.

La **génération enrichie**, ainsi nommée, n'est pas une invention du modèle : c'est une **synthèse contextualisée** à partir d'informations sélectionnées, hiérarchisées et vérifiables. Le LLM agit ici comme un expert qui consulte des extraits, fait des liens et produit une réponse lisible, précise et traçable.

6. Une architecture mutualisée pour déployer des assistants RAG par domaine

Le projet UQAM-GPT vise à transformer l'accès à l'information dans les services universitaires en déployant des assistants conversationnels capables de répondre à des questions complexes, en langage naturel, à partir de documents internes. Pour répondre aux besoins spécifiques de différents groupes d'utilisateurs, le système s'appuie sur une architecture mutualisée, mais déclinée par domaine : **Postdoc**, **Labo**, **Appro**.

Ces déclinaisons répondent à des enjeux bien concrets : l'assistant **Postdoc** permet de s'orienter dans la complexité des exigences administratives, fiscales et contractuelles liées aux stages postdoctoraux. L'assistant **Labo** soutient les étudiants et responsables de laboratoires dans la consultation de procédures, règlements et ressources internes. L'assistant **Appro**, quant à lui, facilite l'accès aux règles encadrant les achats, les appels d'offres, ou encore les processus de commande.

Pour chacun de ces cas, le défi est le même : interroger des documents longs, fragmentés ou hétérogènes, sans exiger de l'utilisateur une lecture exhaustive ou une

Michel Héon PhD
heon.miche@uqam.ca
UQAM

maîtrise des structures administratives. Le système RAG fournit ici une réponse pertinente : il permet d'extraire les passages utiles à la volée, de les soumettre à un modèle de langage, et de produire une réponse contextualisée, claire et sourcée.

Ce chapitre décrit l'architecture technique qui permet de rendre cela possible. Déployée sur Azure, elle repose sur les trois grandes phases du pipeline RAG : **indexer**, **interroger**, **générer**, tout en assurant une spécialisation fonctionnelle par domaine et une gouvernance unifiée.

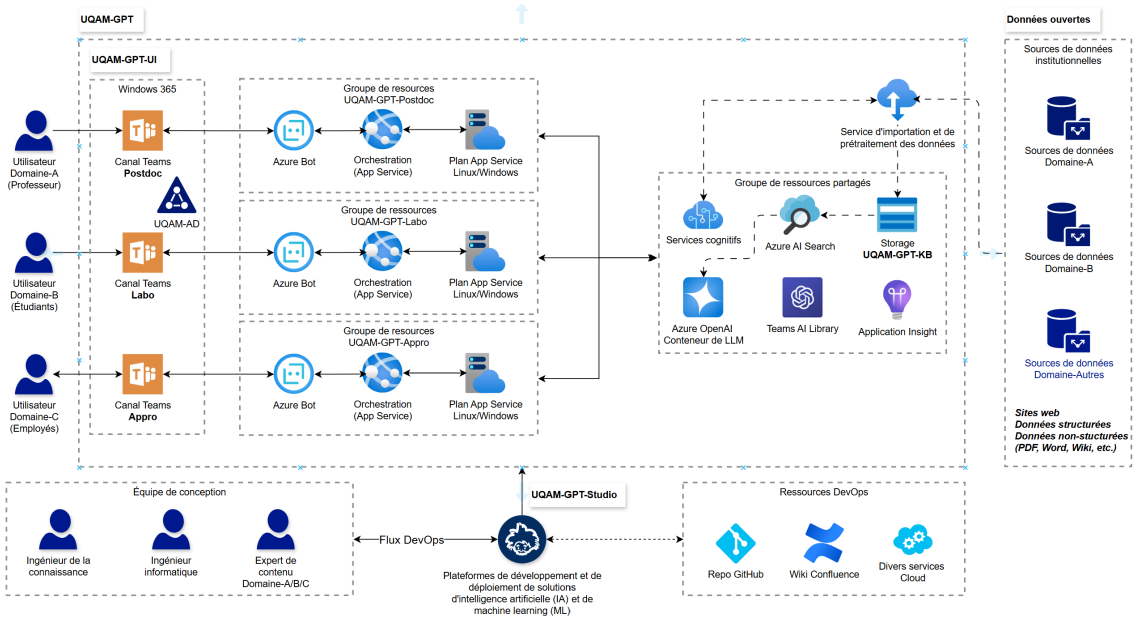

Figure 6 : UQAM-GPT Multi services — Architecture par mutualisation des ressources

La figure 6 illustre la mise en œuvre d'un système RAG à l'échelle institutionnelle. Chaque bloc correspond à une composante fonctionnelle rattachée à l'une des étapes du processus : ingestion documentaire, traitement vectoriel, recherche sémantique ou génération de réponse. Les flèches indiquent les flux de données, la logique d'orchestration et les interactions entre services. Ce schéma sera analysé à travers les trois phases du RAG telles que définies précédemment dans ce document.

Phase 1 — Indexer : transformer les documents en empreintes sémantiques

Tout commence par la collecte des données. À droite de la figure, on retrouve les **sources institutionnelles** (PDF, Word, sites web, wikis, bases de données). Ces documents sont transmis à un **service d'importation** qui applique des traitements cognitifs : extraction de texte, segmentation (chunking), nettoyage.

Michel Héon PhD
heon.miche@uqam.ca
UQAM

Chaque fragment obtenu est transformé en vecteur numérique à l'aide d'un modèle d'**embedding**, puis stocké dans **Azure AI Search**, qui joue ici le rôle de base de connaissances vectorielle. Cette étape correspond au pipeline d'ingestion décrit à la section 5.1 : le corpus est converti en **empreintes sémantiques**, accessibles via des opérations de recherche par similarité

Phase 2 — Interroger : chercher la signification dans l'espace vectoriel

Lorsqu'un utilisateur soumet une question (via un canal Teams), celle-ci est vectorisée, c'est-à-dire projetée dans le même espace sémantique que les documents indexés. Le moteur de recherche (**Azure AI Search**) calcule alors la **proximité cosinus** entre la question et les fragments disponibles.

Les résultats les plus proches sur le plan du sens — et non des mots-clés — sont sélectionnés pour alimenter la suite du raisonnement. Cette logique est implémentée au sein de **groupes de ressources Azure** propres à chaque domaine (Postdoc, Labo, Appro), chacun disposant de :

- son **Azure Bot** (interface conversationnelle),
- son **App Service** (orchestration),
- son propre plan d'exécution (Linux ou Windows).

Chaque domaine est ainsi libre de paramétrer son assistant : corpus dédié, prompt système, rôle joué par le modèle.

Phase 3 — Générer : construire une réponse contextualisée

Les extraits de texte identifiés sont insérés dans un **prompt composite** qui contient également :

- la question de l'utilisateur,
- les consignes de génération (style, ton, structure),
- le rôle à jouer par le LLM (ex. conseiller RH, expert syndical).

Ce prompt est ensuite transmis à **Azure OpenAI**, qui mobilise ses capacités de raisonnement pour produire une réponse structurée, formulée en langage naturel, et **ancrée dans les documents fournis** — c'est ce qu'on appelle la génération enrichie (voir section 5.3)

Une spécialisation fonctionnelle sur une base mutualisée

L'architecture UQAM-GPT repose sur un principe simple : mutualiser l'infrastructure technique tout en permettant une spécialisation fonctionnelle par domaine. La base vectorielle, le modèle de langage (LLM) et le stockage des documents sont partagés entre tous les assistants, ce qui réduit les coûts et centralise l'entretien. En revanche,

Michel Héon PhD
heon.miche@uqam.ca
UQAM

chaque domaine — Postdoc, Labo, Appro — conserve ses propres configurations : corpus dédié, logique d'orchestration, rôle conversationnel, ton et consignes de génération. Cette séparation permet d'adapter finement les réponses aux besoins des utilisateurs, tout en bénéficiant d'une plateforme technique robuste et cohérente.

Pilotage technique et gouvernance

Le système UQAM-GPT est piloté depuis un environnement DevOps centralisé, **UQAM-GPT-Studio**, qui regroupe le code (GitHub), la documentation (Confluence) et les pipelines de déploiement continu (CI/CD). La supervision est assurée via **Application Insight**, permettant un suivi des performances et de l'usage.

La gouvernance repose sur une collaboration entre trois pôles : un **ingénieur logiciel** pour l'infrastructure, un **ingénieur de la connaissance** pour la structuration des contenus, et un **expert métier** pour la validation fonctionnelle. Ce trio assure un développement itératif, aligné sur les besoins concrets des utilisateurs.

Récapitulatif des composantes alignées sur le pipeline RAG

L'architecture UQAM-GPT matérialise les trois phases du RAG à travers des services Azure dédiés. La phase **Indexer** s'appuie sur les services cognitifs et Azure Storage pour transformer les documents en vecteurs, puis les indexer dans **Azure AI Search**. La phase **Interroger** utilise **Azure Bot** et **App Service** pour vectoriser les questions et rechercher les passages pertinents dans la base vectorielle. Enfin, la phase **Générer** repose sur **Azure OpenAI**, qui produit une réponse à partir d'un prompt enrichi, restituée à l'utilisateur dans **Microsoft Teams**.

Conclusion

Le déploiement d'un système RAG transforme un amas de documents épars en un assistant intelligent capable de répondre instantanément aux questions les plus pointues, en citant précisément ses sources. Chaque acteur y trouve son compte : les employés gagnent du temps, les experts réduisent les tâches répétitives, les informaticiens disposent d'un système modulaire et observable, et les dirigeants obtiennent des garanties de fiabilité et de retour sur investissement.

Face à la surcharge informationnelle, le RAG apporte une réponse concrète : il connecte les utilisateurs à la connaissance métier sans détour, sans apprentissage préalable, et avec une précision sémantique inégalée. Ce n'est plus l'humain qui cherche dans les documents, c'est l'IA qui va y puiser la bonne clause, la bonne procédure ou le bon article — en quelques secondes.

Pour les organisations, la recommandation est claire :

Michel Héon PhD
heon.miche@uqam.ca
UQAM

1. **Auditez vos gisements documentaires** : identifiez les corpus riches, structurés et à forte valeur.
2. **Lancez un projet pilote RAG** dans un service ciblé (RH, juridique, approvisionnement…).
3. **Mesurez les gains concrets** (temps économisé, fiabilité accrue, retour des utilisateurs).
4. **Étendez progressivement l'approche**, en capitalisant sur ce premier succès.

Le RAG n'est pas une technologie parmi d'autres : c'est un nouveau paradigme dans l'accès à la connaissance. Les organisations qui s'en dotent aujourd'hui seront les premières à transformer leur capital documentaire en avantage stratégique.

Lexique

Agent conversationnel : Interface qui permet à l'utilisateur de poser des questions en langage naturel (ex. : chatbot, canal Teams). Il relaie la demande vers le système RAG et affiche la réponse générée.

Azure AI Search : Service cloud utilisé comme base vectorielle pour stocker et interroger les représentations numériques des documents (vecteurs d'empreinte sémantique).

Azure OpenAI : Service d'inférence hébergeant un modèle de langage (LLM) utilisé pour générer des réponses contextualisées à partir du prompt enrichi.

Base vectorielle : Index sémantique dans lequel sont stockés les vecteurs représentant les fragments de documents. Elle permet la recherche par similarité de sens.

Chunking (segmentation) : Découpage des documents en unités textuelles cohérentes (paragraphes, phrases) destinées à être vectorisées.

Empreinte sémantique : Représentation vectorielle d'un fragment de texte capturant son sens dans un espace abstrait.

Génération enrichie : Production d'une réponse par un LLM à partir d'un prompt composite incluant question, contexte documentaire et consignes.

Indexation : Processus d'analyse, de transformation en vecteurs et de stockage des documents dans une base vectorielle.

LLM (Large Language Model) : Modèle d'intelligence artificielle entraîné sur de grandes quantités de texte. Utilisé ici pour raisonner sur des extraits documentaires et produire des réponses.

Michel Héon PhD
heon.miche@uqam.ca
UQAM

Pipeline RAG : Enchaînement structuré des trois phases du système RAG : indexer, interroger, générer.

Prompt composite : Texte structuré envoyé au LLM contenant la question, les extraits documentaires sélectionnés, le rôle attendu, et des instructions précises.

Proximité cosinus : Mesure mathématique utilisée pour évaluer la similarité entre deux vecteurs dans un espace sémantique.

RAG (Retrieval-Augmented Generation) : Architecture d'IA qui combine une recherche sémantique dans des documents internes avec la génération de réponse par un modèle de langage.

Tokenisation : Découpage d'un texte en unités élémentaires (tokens), souvent des sous-mots, utilisés comme base pour la vectorisation.

Vectorisation : Transformation des tokens en vecteurs numériques porteurs de sens, permettant la comparaison sémantique entre textes.

--FIN--

Michel Héon PhD
heon.miche@uqam.ca
UQAM
07-05-2025